



Activité Numéro 5

Objectif : Utilisation d'une base de données locale SqfLite

L'objectif de cette activité est de vous familiariser avec la création d'une application Flutter de gestion de données, en mettant l'accent sur les opérations CRUD (Create, Read, Update, Delete). Il s'agit essentiellement de maîtriser la gestion des données via la bibliothèque **sqflite** pour gérer une base de données SQL Lite.

Ressources : Ressources du module
Recherche sur Internet

Type de l'activité : Activité individuelle

Outils de communication : Forum

Résultat attendu : Un fichier compressé (le code de l'application flutter)

Durée : 04 heures

Evaluation : Par le Tuteur

Consignes :

L'objectif de cette activité est de compléter votre première application Magazine infos pour que l'éditeur en chef puisse voir les informations relatives à tous ses rédacteurs. Vous êtes chargé(e) de développer une application de gestion des rédacteurs. Les rédacteurs sont des personnes qui écrivent du contenu pour Magazine infos. Vous devez créer une application Flutter qui permettra d'ajouter, de modifier, de supprimer et d'afficher la liste des rédacteurs.

Votre application est censée avoir un rendu conforme à la figure suivante :



Cours développement Mobile

Niveau Intermédiaire

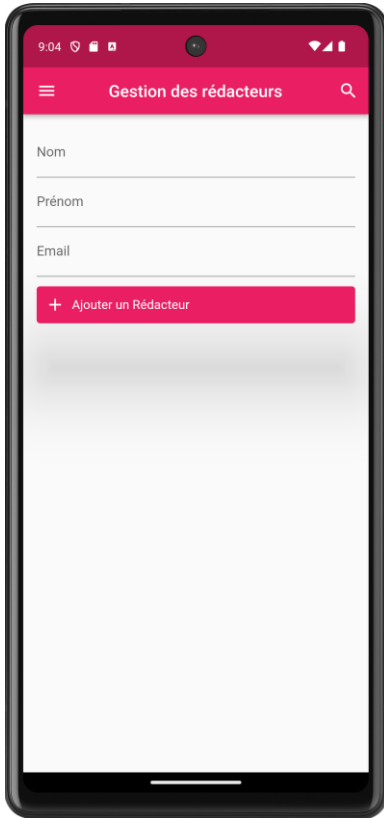


Figure 1. Ecran principal de votre application de gestion des rédacteurs

Vous allez suivre les instructions suivantes pour réaliser l'activité :

1. D'abord vous devriez créer une classe **Redacteur** avec les attributs suivants : **id** (de type entier et clé primaire avec auto incrémentation), **nom** (de type chaîne de caractères), **prenom** (de type chaîne de caractères), et **email** (de type chaîne de caractères).
2. Créez la classe **Redacteur** pour créer et manipuler les objets rédacteurs. Cette classe constitue le modèle de votre application. Vous pouvez la mettre sous un dossier appelé **Modele** sous le dossier **lib** de votre projet. Vous pouvez respecter cette définition pour votre classe :

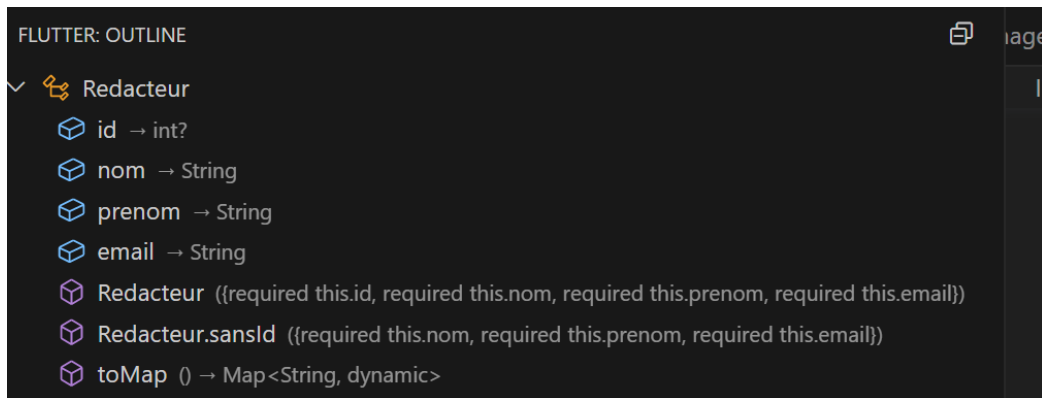


Figure 2. Définition de la classe Redacteur

La classe comporte deux constructeurs **un avec tous les attributs** et **un sans l'attribut id**. La méthode **toMap()** permet de convertir un objet rédacteur en un **Map**.

2. Créez une classe **DatabaseManager** pour gérer la base de données des rédacteurs et effectuer des opérations sur la base de données avec sqflite. Cette classe devrait avoir les méthodes suivantes :

- **initialisation** : pour initialiser la base de données SQLite avec une table **redacteurs**.
- **getAllRedacteurs** : pour récupérer tous les rédacteurs de la base de données.
- **insertRedacteur** : pour ajouter un rédacteur à la base de données.
- **updateRedacteur** : pour mettre à jour les informations d'un rédacteur.
- **deleteRedacteur** : pour supprimer un rédacteur de la base de données.

Vous êtes libre d'utiliser l'un des méthodes pour la manipulation de **sqflite** que ce soit avec l'utilisation de requêtes SQL directes ou l'exploitation d'une API de haut niveau pour les opérations SQL brutes (**RawSQL**). Aussi n'oubliez pas tous les imports nécessaires et la déclaration du dépendance dans le fichier **pubspec.yaml**.

3. Votre **main()** est censé invoquer en premier lieu un **StatelessWidget** appelé **MonApplication**. Ce Widget retourne un **MaterialApp** et respectera cette arborescence :

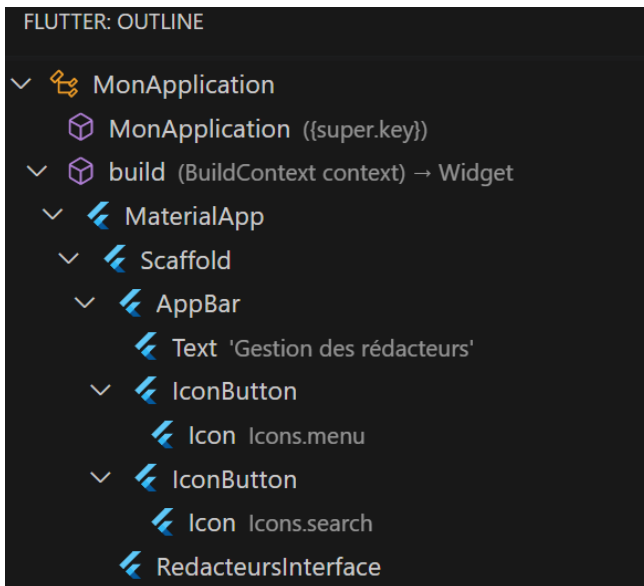


Figure 3. Arborescence de Widgets de MonApplication

3. Créez le Widget **RedacteurInterface** pour gérer les rédacteurs. Ce Widget étend la classe StatefulWidget. **RedacteurInterface** doit comporter les éléments suivants :

- Un champ de texte pour saisir le nom d'un rédacteur. Ce champ de texte doit être contrôlé par un contrôleur associé.

Voilà comment on définit un champ de texte où l'utilisateur peut saisir du texte :

```
TextField(
  controller: _nomController,
  decoration: const InputDecoration(labelText: 'Nom'),
),
```

Ainsi :

- **TextField** : Il s'agit du widget principal qui affiche un champ de texte interactif où l'utilisateur peut saisir du texte.
- **controller: _nomController** : cette propriété lie le champ de texte à un contrôleur de texte (TextEditingController) nommé **_nomController**. Un contrôleur de texte permet de gérer le contenu du champ de texte, d'accéder à sa valeur actuelle et de le modifier.

- **decoration: const InputDecoration(labelText: 'Nom')** : cette propriété définit la décoration du champ de texte. Dans cet exemple, il affiche un texte "Nom" au-dessus du champ de texte pour indiquer à l'utilisateur ce qu'il devrait saisir.
- Un champ de texte pour saisir l'adresse e-mail d'un rédacteur. Prévoyez aussi un contrôleur associé.
- Un bouton "Ajouter un Rédacteur" avec une icône d'ajout (voir exemple d'ajout) :

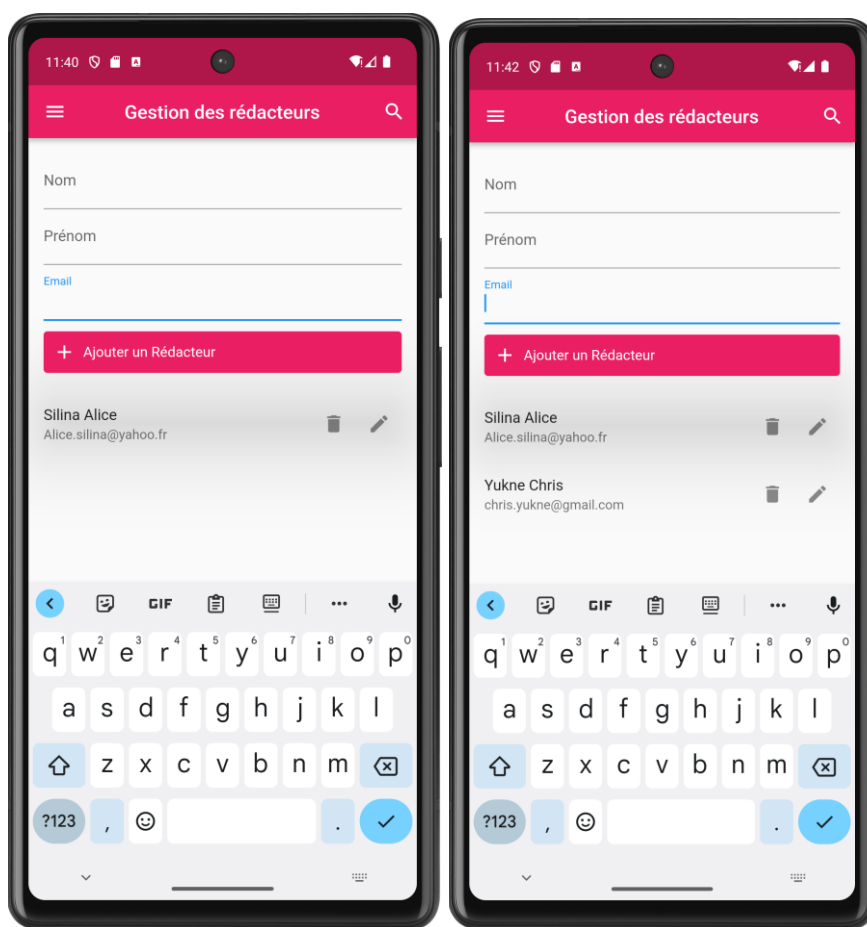


Figure 4. Ajout d'un rédacteur

- Une liste de rédacteurs affichée à l'écran, avec la possibilité de les modifier ou de les supprimer.

3.1. Implémentez la logique pour ajouter un rédacteur à la base de données lorsque le bouton "Ajouter un Rédacteur" est cliqué.

3.2. Implémentez la fonctionnalité de modification en affichant une boîte de dialogue lorsqu'un utilisateur appuie sur l'icône de modification. La boîte de dialogue devrait permettre de modifier les informations du rédacteur sélectionné.

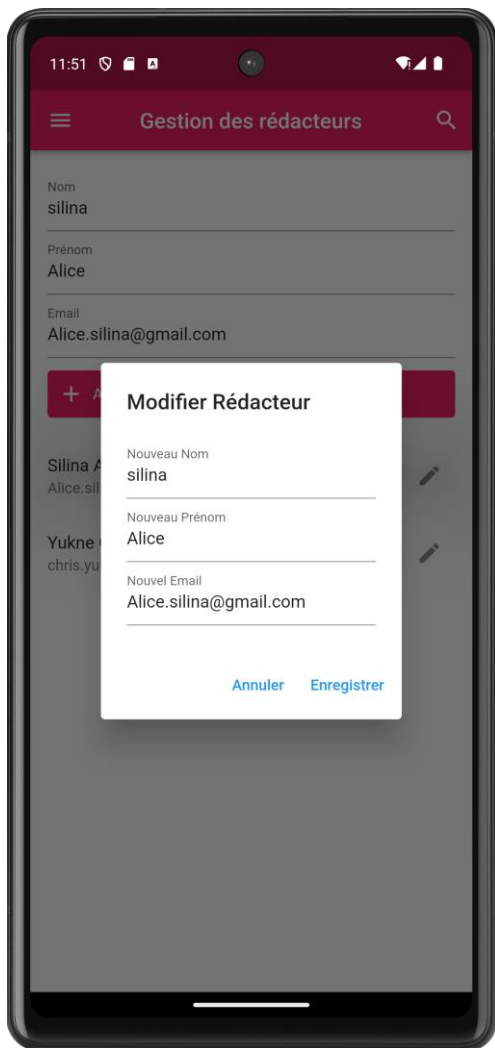


Figure 5. Modification des informations d'un rédacteur

3.3. Affichez la liste des rédacteurs existants dans un **ListView.builder**. Chaque élément de la liste doit afficher le nom, le prénom et l'email du rédacteur. Ajoutez des icônes pour permettre la modification et la suppression de chaque rédacteur.

3.4. Implémentez la fonctionnalité de suppression en affichant une boîte de dialogue de confirmation lorsqu'un utilisateur appuie sur l'icône de suppression. Cette boîte de dialogue devrait demander à l'utilisateur s'il est sûr de vouloir supprimer le rédacteur.

Note :



Cours développement Mobile

Niveau Intermédiaire

Veillez à ce qu'à chaque ouverture de votre application, la liste des rédacteurs déjà enregistrée est affichée. Pour ce faire, il faut se baser sur la méthode **initState()**. Cette méthode est une méthode spéciale que vous pouvez utiliser dans les widgets d'interface utilisateur de Flutter pour effectuer des initialisations nécessaires au moment où le widget est créé. Elle est appelée automatiquement par Flutter lors de la création du widget, juste avant que le widget ne soit ajouté à l'arborescence des widgets.